

Static Analysis for LLM Code Repair: A Methodology Study

Anonymous

Abstract

Iterative LLM code repair relies primarily on execution feedback—test results and stack traces. Static analysis offers complementary signal: mechanistic explanations of *why* code is problematic, not just *where* it fails. Prior work shows static analysis improves code quality metrics (security, readability), but impact on functional correctness (pass@k) remains unmeasured.

We present a methodology study for evaluating static analysis in LLM code repair. We develop a gate-based hypothesis validation framework, implement a pylint analysis pipeline for HumanEval, and establish baseline measurements. Our existence gate tests whether static analyzers produce actionable warnings on benchmark code—a prerequisite for the intervention to have signal.

Key finding: Canonical HumanEval solutions exhibit only 9.15% pylint warning rate (15/164 problems with ≥ 1 warning, 23 total warnings across 9 categories). This makes canonical solutions invalid proxies for LLM-generated code, which typically contains more issues. The gate correctly halted downstream experiments.

Contributions: (1) First baseline of static warning rates on HumanEval canonical solutions; (2) Validated pylint pipeline for code generation research; (3) Gate-based validation methodology; (4) Identification of canonical-vs-LLM proxy limitation.

The hypothesis that static analysis improves pass@k remains untested. We provide the validated infrastructure and baseline for future work with LLM API access.

1 Introduction

Large language models (LLMs) have transformed code generation, achieving remarkable performance on benchmarks like HumanEval and MBPP. Iterative repair loops—where models refine their output based on feedback—have emerged as a key technique for improving functional correctness. The dominant paradigm, exemplified by Self-Debug, relies on execution feedback: test results and stack traces guide the model toward working solutions.

Static analysis offers an alternative signal. Tools like pylint and mypy detect potential issues—undefined variables, type mismatches, unreachable code—before execution. Unlike execution traces that show *where* code fails, static warnings explain *why* the code is problematic. This mechanistic feedback could enable more targeted repairs.

The gap. Prior work has shown static analysis improves code *quality* metrics. Blyth et al. demonstrated reductions in security issues (40% $\rightarrow$ 13%) and readability problems (80% $\rightarrow$ 11%) when integrating static feedback into LLM pipelines. However, no study has measured the impact on *functional correctness*—the pass@k metrics that matter for code generation benchmarks.

Our question. Does integrating static analyzer feedback into iterative LLM code repair improve pass@k compared to execution-only feedback?

This paper. We present a methodology study that establishes the groundwork for answering this question. We develop a hypothesis validation framework with gate-based testing, implement a pylint analysis pipeline for HumanEval, and provide baseline measurements. Our existence gate—testing whether static analyzers produce actionable warnings on benchmark code—reveals a critical methodological insight: canonical solutions exhibit only 9.15% warning rates, making them invalid proxies for LLM-generated code.

Contributions:

1. First baseline measurement of static warning rates on HumanEval canonical solutions (9.15%, 23 warnings across 9 categories)
2. Validated pylint analysis pipeline for code generation research (reusable components)

3. Gate-based hypothesis validation methodology that correctly catches methodology limitations
4. Identification of proxy limitation: canonical solutions $\neq$ LLM output for static analysis evaluation

We frame this as a methodology contribution. The hypothesis that static analysis improves pass@k remains untested—but we provide the validated pipeline and baseline for future work with LLM API access.

2 Related Work

2.1 Iterative Code Repair

Self-Debug [4] introduced execution-feedback loops for LLM code generation, using test results and stack traces to guide iterative refinement. Self-Refine [5] generalized this to broader refinement tasks using LLM self-feedback. These methods establish execution feedback as the baseline for repair loops.

LDB (LLM Debugger) [6] extended this paradigm with block-by-block verification, improving localization of errors. However, all these approaches rely on execution traces—they know *where* code fails but not *why*.

2.2 Static Analysis for LLMs

Blyth et al. [2] represent the closest prior work. They integrated static analysis feedback (pylint, bandit) into LLM pipelines and demonstrated improvements in code quality:

- Security issues: 40% $\rightarrow$ 13%
- Readability problems: 80% $\rightarrow$ 11%

Critically, they did not measure functional correctness (pass@k). Their contribution establishes that LLMs can interpret static warnings—but whether this improves benchmark performance remains unknown.

2.3 The Gap We Address

Table 1: Comparison of feedback approaches in prior work.

Prior Work	Feedback Type	Metric	Pass@k?
Self-Debug	Execution	Functional correctness	✓
Self-Refine	LLM self-feedback	Task completion	✓
Blyth et al.	Static analysis	Quality (security, readability)	×
This work	Static + Execution	Methodology baseline	Pending

The research gap is clear: no study measures static analysis impact on pass@k. We provide the methodology and baseline to enable this evaluation.

2.4 Benchmarks

HumanEval [3] (164 problems) and MBPP [1] (974 problems) are standard benchmarks for code generation. Pass@k metrics measure functional correctness—the fraction of problems solved within k attempts. We focus on HumanEval as the more widely-used benchmark, with MBPP extension as future work.

3 Methodology

3.1 Hypothesis Structure

We formulate our research question as a structured hypothesis with testable predictions:

Core Hypothesis (H-StaticFeedback-v1): Under iterative LLM code repair on HumanEval/MBPP, integrating static analyzer feedback (pylint/mypy) alongside execution feedback improves pass@k compared to execution-only feedback, because static analysis provides mechanistic error explanations.

Causal Mechanism:

1. Static analyzer produces warnings on LLM-generated code
2. LLM receives mechanistic feedback (why, not just where)
3. Targeted repairs lead to higher pass@k

Key Assumptions:

- A1: Static analyzers produce meaningful warnings on LLM code
- A2: LLMs can interpret static warnings into fixes
- A3: Static and execution feedback catch non-overlapping errors
- A4: HumanEval contains problems with static-detectable errors
- A5: Pylint severity filtering (disable conventions) is effective

3.2 Gate-Based Validation

We employ a hierarchical gate structure:

Table 2: Gate structure for hypothesis validation.

Gate	Type	Hypothesis	Criterion
h-e1	MUST_WORK	Existence	$\geq 30\%$ of problems have actionable warnings
h-m1	MUST_WORK	Mechanism	LLM interprets warnings into fixes
h-m2	SHOULD_WORK	Mechanism	Static/execution non-overlapping
h-c1	SHOULD_WORK	Comparison	Improvement stratified by warnings

MUST_WORK gates block downstream experiments if failed. This design catches methodology issues early.

3.3 Experimental Design

Variables:

- Independent: Feedback type (execution-only vs. static+execution)
- Dependent: pass@1, pass@5, pass@10
- Controlled: Base LLM, iteration budget (5), prompt format, static analyzer (pylint)

Dataset: HumanEval (164 problems)

Static Analysis Configuration:

```
pylint_config = {
    "disabled_categories": ["C", "R"], # Convention, Refactoring
    "enabled_types": ["error", "warning"],
    "output_format": "json"
}
```

3.4 Existence Gate Protocol

Before full pass@k evaluation, we validate the existence assumption: do static analyzers produce actionable warnings on benchmark code?

Protocol:

1. Load all HumanEval problems
2. Extract code (canonical solutions or LLM generations)
3. Run pylint with configured filters
4. Count problems with ≥ 1 actionable warning
5. Gate passes if $\text{warning_rate} \geq 30\%$

Rationale: If most solutions have zero warnings, static analysis provides no signal—the intervention is null. We set the 30% threshold based on practical considerations: below this rate, fewer than 1 in 3 problems would receive static feedback, limiting the intervention’s statistical power and practical utility. This threshold is conservative; lower rates (e.g., 20%) might still enable meaningful experiments but would require larger sample sizes to detect effects.

3.5 Metrics

- **Warning Rate:** Fraction of problems with ≥ 1 actionable warning
- **Warnings per Problem:** Average actionable warnings
- **Warning Distribution:** Breakdown by pylint code

4 Experiments

4.1 Existence Gate Experiment (h-e1)

We test the first causal mechanism step: do static analyzers produce actionable warnings on benchmark code?

Setup:

- Dataset: HumanEval (164 problems, full test set)
- Code source: Canonical solutions (proxy for LLM output due to API unavailability)
- Analyzer: pylint 3.x with `-disable=C,R`
- Gate threshold: $\geq 30\%$ of problems with ≥ 1 warning

Rationale for Proxy: Without OpenAI/Claude API access, we used canonical solutions as a lower-bound proxy. Canonical solutions are well-written human code; LLM-generated code typically exhibits more issues.

4.2 Implementation

Pipeline Components:

1. `humaneval_loader.py` — Loads HumanEval problems from HuggingFace
2. `static_analyzer.py` — Runs pylint, parses JSON output, filters by severity
3. `metrics.py` — Aggregates warning counts, computes rates

Pylint Configuration:

```
pylint --output-format=json --disable=C,R <code_file>
```

4.3 Artifacts

Table 3: Generated artifacts from the existence gate experiment.

Artifact	Description
metrics.json	Aggregated warning statistics
pylint_results.json	Per-problem pylint output
warning_distribution.png	Histogram of warnings per problem
warning_type_breakdown.png	Pie chart by category
gate_metrics.png	Threshold vs. actual comparison
top_warning_codes.png	Top 10 warning codes

5 Results

5.1 Existence Gate Outcome

Gate Result: FAIL

Table 4: Existence gate metrics.

Metric	Value	Threshold
Warning Rate	9.15%	$\geq 30\%$
Problems with ≥ 1 Warning	15 / 164	—
Total Warnings	23	—
Avg Warnings/Problem	0.14	—

The existence gate failed: only 9.15% of canonical solutions had actionable pylint warnings, well below the 30% threshold.

5.2 Warning Distribution

Table 5: Distribution of warning types across HumanEval canonical solutions.

Warning Code	Count	Description
bad-indentation	12	Incorrect indentation
unused-import	3	Imported but unused
bare-except	2	Catching all exceptions
unused-variable	1	Defined but unused
unnecessary-semicolon	1	Trailing semicolon
redefined-builtin	1	Shadowing builtin
pointless-string-statement	1	String with no effect
unreachable	1	Code after return
eval-used	1	Using eval()

5.3 Analysis

The low warning rate on canonical solutions is **expected behavior**. HumanEval canonical solutions are curated, high-quality reference implementations written by humans. They represent a floor, not a ceiling, for static warning rates.

Key insight: Canonical solutions $\neq$ LLM-generated code. This proxy does not test the hypothesis.

5.4 Gate Mechanism Validation

The MUST_WORK gate correctly halted the pipeline:

- h-e1 FAIL $\rightarrow$ h-m1, h-m2, h-c1 blocked
- No downstream experiments wasted on invalid proxy
- Methodology limitation identified before full evaluation

6 Discussion

6.1 Why the Proxy Failed

Canonical HumanEval solutions are carefully crafted reference implementations. They pass all tests by design and follow Python conventions. LLM-generated code—especially early attempts in a repair loop—contains more issues: undefined variables, type mismatches, incomplete logic.

Using canonical solutions as an LLM proxy conflates the reference answer with the model’s attempts. The 9.15% warning rate reflects human code quality, not LLM generation behavior.

6.2 What the Baseline Tells Us

Despite the proxy limitation, our results provide value:

1. **Floor established:** Canonical solutions have $\sim 9\%$ warning rate. LLM code should exceed this, providing room for the static analysis intervention to act.
2. **Pipeline validated:** The pylint analysis infrastructure works. Components are reusable for future work with LLM API access.
3. **Warning types identified:** The 9 warning categories are actionable for code repair. They explain *why* code is problematic.

6.3 Gate Design Implications

The MUST_WORK gate served its purpose. Had we skipped the existence check and proceeded to full pass@k evaluation, we would have:

- Wasted compute on the wrong code source
- Produced results that don’t test the hypothesis
- Potentially drawn incorrect conclusions

Gate-based validation catches methodology issues early.

6.4 Limitations

Primary: No LLM-generated code tested. API access unavailable during this study.

Secondary:

- Single dataset (HumanEval only, not MBPP)
- Single static analyzer (pylint, not mypy)
- Single severity filter configuration

Framing: This is a methodology study establishing baseline and validating infrastructure—not a full hypothesis test.

6.5 Path Forward

To complete the hypothesis evaluation:

1. **Re-run h-e1 with LLM API:** Generate code using GPT-4 or Claude, measure warning rates
2. **If h-e1 passes:** Proceed to h-m1 (LLM interprets warnings) and h-m2 (non-overlapping errors)
3. **Full evaluation:** Compare pass@k between static+execution vs. execution-only conditions

The pipeline, baseline, and methodology are ready. The missing ingredient is LLM API access.

7 Conclusion

We asked whether static analyzers can improve LLM code repair. The answer remains open—but we now have the methodology to find it.

What we found: Canonical HumanEval solutions exhibit only 9.15% pylint warning rates. This makes them invalid proxies for LLM-generated code. Our MUST_WORK gate correctly identified this limitation before downstream experiments wasted resources.

What we built: A validated pylint analysis pipeline for code generation research, reusable components for HumanEval loading and static analysis, and a gate-based hypothesis validation framework.

What we learned: Gate-based validation works. By testing existence assumptions before full experiments, we catch methodology issues early.

The path forward: Re-run the existence gate with actual LLM generations. If warning rates exceed 30%, proceed to mechanism validation and full pass@k comparison.

Broader implication: Before evaluating any LLM intervention, validate that the intervention has signal. Canonical solutions are not LLM output—a lesson applicable beyond static analysis to any benchmark-based evaluation.

We contribute methodology, not conclusions. The question of whether static analysis improves pass@k is worth answering—and we provide the tools to answer it.

References

- [1] Jacob Austin et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. MBPP benchmark (974 problems).
- [2] Harrison Blyth et al. Static analysis as a feedback loop for LLM code generation. *arXiv preprint arXiv:2508.14419*, 2025. Security 40%→13%, readability 80%→11%; no pass@k.
- [3] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. HumanEval benchmark (164 problems).
- [4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023. Execution-only feedback baseline.
- [5] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. *NeurIPS*, 2023. LLM self-feedback framework.
- [6] Li Zhong et al. LDB: A large language model debugger via verifying runtime execution step-by-step. *arXiv preprint arXiv:2402.16906*, 2024. Block-by-block verification.

A Figures

- Figure 1: Warning type breakdown (`warning_type_breakdown.png`)
- Figure 2: Warning distribution histogram (`warning_distribution.png`)
- Figure 3: Gate metrics comparison (`gate_metrics.png`)
- Figure 4: Top warning codes (`top_warning_codes.png`)

Note: Figures referenced above are available in the experiment artifacts directory. Due to the methodology focus of this paper, figures serve as supplementary material rather than primary evidence.